

AI Implementation Brief — Zombie Platform Shooter

You are helping build a Python/Pygame game called **Zombie Platform Shooter**.

The goal is to create a simple, playable 2D wave-survival platform shooter. The game should be built with rectangles first, then later replace the player, zombies, bullet, and optional platform tile with static PNG images.

Do not add animations, scrolling camera, bosses, complex pathfinding, projectiles from enemies, multiple maps, save files, or multiplayer in version 1.

1. Game Summary

Build a 2D zombie-themed platform shooter in Python using Pygame.

The player is trapped in a single static platform arena. Zombies spawn in waves and try to kill the player by touching them. The player moves, jumps, aims with the mouse, and shoots bullets. After each completed level, the player chooses one of three random upgrade cards. The game has 10 levels. Clearing level 10 shows a victory screen. Reaching 0 HP shows a game-over screen.

2. Core Technical Requirements

Use:

```
Python
Pygame
1280 x 720 window
60 FPS using clock.tick(60)
Static camera
Frame-based movement
```

The first working version must use rectangles. After the rectangle version works, replace these rectangles with static PNG images:

```
player
walker_zombie
tank_zombie
flying_zombie
bullet
platform tile optional
background optional
```

No animations are required for version 1.

3. Project Folder Structure

Create this structure:

```
zombie_platform_shooter/  
  main.py  
  settings.py  
  game.py  
  player.py  
  bullet.py  
  enemy.py  
  platforms.py  
  level_manager.py  
  upgrade_manager.py  
  ui.py  
  assets/  
    images/  
      player.png  
      walker_zombie.png  
      tank_zombie.png  
      flying_zombie.png  
      bullet.png  
      platform.png  
      background.png  
    sounds/
```

The PNG files are optional at first. The game must still work if the PNGs are missing by drawing rectangles instead.

4. Asset Rules

Use separate PNG files rather than one large sprite sheet.

Expected asset paths:

```
assets/images/player.png  
assets/images/walker_zombie.png  
assets/images/tank_zombie.png  
assets/images/flying_zombie.png  
assets/images/bullet.png  
assets/images/platform.png  
assets/images/background.png
```

Use PNGs for the player, Walker Zombie, Tank Zombie, Flying Zombie, and bullet.

Do not create the whole level as one big PNG image. The world and platforms should be coded directly as rectangles in Python. This keeps collision simple and easy to edit.

The clean rule is:

```
Visuals = images  
Physics and collision = pygame.Rect objects
```

Example:

```
self.image = pygame.image.load("assets/images/  
walker_zombie.png").convert_alpha()  
self.rect = self.image.get_rect(topleft=(x, y))
```

Movement and collision must use `self.rect`, not the pixels of the image.

If a PNG is not found, draw a coloured rectangle instead.

5. Controls

Use these exact controls:

```
Move left: A or Left Arrow  
Move right: D or Right Arrow  
Jump: Space  
Aim: Mouse position  
Shoot: Left Mouse Click  
Pause: Esc  
Select upgrade: Mouse click on card or number keys 1, 2, 3  
Restart after death: R
```

The player can shoot while standing, moving, or jumping.

Bullets travel toward the mouse cursor at the moment of firing.

Bullets do not collide with platforms in version 1.

6. Game States

Use a game state variable, preferably strings or an Enum:

```
MAIN_MENU  
PLAYING  
UPGRADE_SELECT  
PAUSED  
GAME_OVER  
VICTORY
```

Only update gameplay during `PLAYING`.

During `UPGRADE_SELECT`, `PAUSED`, `GAME_OVER`, and `VICTORY`, zombies, bullets, and player physics should not keep updating.

State transitions:

```
MAIN_MENU -> PLAYING when Start is clicked
PLAYING -> UPGRADE_SELECT when all zombies in the current wave are dead
UPGRADE_SELECT -> PLAYING after the player chooses an upgrade
PLAYING -> GAME_OVER when player health <= 0
PLAYING -> PAUSED when Esc is pressed
PAUSED -> PLAYING when Esc is pressed again
PLAYING -> VICTORY when level 10 is cleared
GAME_OVER -> restart when R is pressed
VICTORY -> restart when R is pressed
```

7. World and Platform Design

Use one static arena. No scrolling. No multiple maps. No hazards. No moving platforms.

Screen:

```
SCREEN_WIDTH = 1280
SCREEN_HEIGHT = 720
FPS = 60
```

Use these exact platforms:

```
PLATFORMS = [
    pygame.Rect(0, 660, 1280, 60),      # ground floor
    pygame.Rect(160, 500, 270, 24),     # left platform
    pygame.Rect(505, 380, 270, 24),     # middle platform
    pygame.Rect(850, 500, 270, 24),     # right platform
    pygame.Rect(490, 250, 300, 24),     # high platform
]
```

Draw platforms as grey rectangles first.

Later, optionally draw `assets/images/platform.png` stretched or tiled over each platform rectangle.

Use a plain dark background first. Optionally later draw `assets/images/background.png`.

8. Spawn Zones

Walker Zombies and Tank Zombies spawn on the ground near the left or right edge.

Flying Zombies spawn from upper screen edges.

Use:

```
Left ground spawn: x = 40 to 120
Right ground spawn: x = 1160 to 1240
Flying spawn: left edge, right edge, or top edge
```

Spawn zombies gradually rather than all at once.

Use:

```
SPAWN_INTERVAL_MIN = 1.2
SPAWN_INTERVAL_MAX = 2.0
```

Only mark the wave as complete when:

```
all planned zombies have spawned
and
all active zombies are dead
```

9. Player Stats

Starting player stats:

```
PLAYER_MAX_HEALTH = 100
PLAYER_START_HEALTH = 100
PLAYER_SPEED = 5
PLAYER_JUMP_STRENGTH = 15
GRAVITY = 0.8
MAX_FALL_SPEED = 20
PLAYER_BASE_DAMAGE = 10
PLAYER_FIRE_COOLDOWN = 0.35
PLAYER_BULLET_SPEED = 12
HURT_INVINCIBILITY = 0.5
PLAYER_WIDTH = 40
PLAYER_HEIGHT = 60
```

Player damage formula:

```
actual_damage = PLAYER_BASE_DAMAGE * damage_multiplier
```

Initial:

```
damage_multiplier = 1.0
```

The player's health does not reset between levels unless healed by upgrades.

10. Player Movement Rules

Apply gravity every frame during gameplay:

```
self.vy += GRAVITY  
self.vy = min(self.vy, MAX_FALL_SPEED)
```

Jump only if on the ground:

```
if jump_pressed and self.on_ground:  
    self.vy = -PLAYER_JUMP_STRENGTH
```

Set `on_ground = False` at the start of each physics update. Platform collision sets it back to `True` if the player lands.

The player can move left/right in the air.

Clamp the player inside screen boundaries.

11. Bullet Rules

Bullets spawn from the player's center and travel toward the mouse cursor.

Use normalized direction:

```
dx = mouse_x - player.rect.centerx  
dy = mouse_y - player.rect.centery  
dist = (dx**2 + dy**2) ** 0.5  
  
if dist > 0:  
    vx = (dx / dist) * player.bullet_speed  
    vy = (dy / dist) * player.bullet_speed
```

Bullets:

deal damage to one enemy
disappear after hitting an enemy
do not collide with platforms
despawn when leaving the screen

12. Zombie Enemy Types

All enemies are zombies.

Enemies only damage the player by touching them.

Enemies do not collide with each other.

No enemy projectiles in version 1.

Walker Zombie

Stats:

WALKER_HP = 40
WALKER_SPEED = 2.0
WALKER_DAMAGE = 10

Rules:

Ground enemy
Appears from level 1
Moves horizontally toward the player
Can jump
Collides with platforms
Dies in 4 normal bullet hits
Deals 10 contact damage

Use simple timer-based jumping in version 1:

If the player is above the Walker Zombie, the Walker can jump every 2-3 seconds.

Tank Zombie

Stats:

```
TANK_HP = 100
TANK_SPEED = 1.0
TANK_DAMAGE = 25
```

Rules:

```
Ground enemy
Introduced at level 3
Moves slower than Walker
Has much more health
Can jump, but feels heavier
Collides with platforms
Dies in 10 normal bullet hits
Deals 25 contact damage
```

Use the same jump logic as Walker but slower/heavier:

```
Jump every 3-4 seconds if the player is above it.
Use slightly smaller jump strength than Walker if needed.
```

Flying Zombie

Stats:

```
FLYING_HP = 30
FLYING_SPEED = 2.6
FLYING_DAMAGE = 8
```

Rules:

```
Flying enemy
Introduced at level 6
Ignores gravity
Ignores platforms
Can fly through platforms
Moves directly toward the player
Dies in 3 normal bullet hits
Deals 8 contact damage
```

Flying movement:

```
dx = player.rect.centerx - self.rect.centerx
dy = player.rect.centery - self.rect.centery
dist = (dx**2 + dy**2) ** 0.5
```



```
if dist > 0:
    self.rect.x += (dx / dist) * self.speed
    self.rect.y += (dy / dist) * self.speed
```

13. Collision Rules

Use rectangles for all collision.

Collision table:

```
Player vs platforms:
    Player stands on platforms and cannot fall through.

Walker/Tank vs platforms:
    Ground zombies collide with platforms and ground.

Flying Zombie vs platforms:
    No collision. Flying Zombie passes through freely.

Enemy vs enemy:
    No collision. Enemies may overlap.

Bullet vs enemy:
    Bullet deals damage and disappears.

Bullet vs platform:
    No collision in version 1.

Player vs enemy:
    Player takes contact damage, then gets 0.5 seconds of invincibility.

Enemy HP <= 0:
    Remove enemy immediately.

Player HP <= 0:
    Enter GAME_OVER immediately.
```

Platform collision method:

1. Move horizontally.
2. Resolve horizontal collisions.
3. Move vertically.
4. Resolve vertical collisions.
5. Set `on_ground = True` only when landing on a platform.

Do not move diagonally and resolve both axes at once.

14. Level Table

Use this exact 10-level table. Do not use enemy stat multipliers in version 1.

```
LEVELS = [
    {"walkers": 3, "tanks": 0, "flyers": 0},
    {"walkers": 5, "tanks": 0, "flyers": 0},
    {"walkers": 5, "tanks": 1, "flyers": 0},
    {"walkers": 6, "tanks": 1, "flyers": 0},
    {"walkers": 6, "tanks": 2, "flyers": 0},
    {"walkers": 5, "tanks": 1, "flyers": 2},
    {"walkers": 6, "tanks": 1, "flyers": 3},
    {"walkers": 7, "tanks": 2, "flyers": 3},
    {"walkers": 8, "tanks": 2, "flyers": 4},
    {"walkers": 7, "tanks": 2, "flyers": 5},
]
```

Level 10 ends the game with a victory screen after all enemies are killed.

15. Upgrade System

After each completed level, show three random upgrade cards.

The player chooses one.

All upgrades stack infinitely.

The same upgrade should not appear twice on the same upgrade screen.

Use:

```
random.sample(upgrade_pool, 3)
```

Upgrade cards:

```
UPGRADES = [
    {
        "name": "Bigger Heart",
        "description": "+25 max HP and heal 25 HP",
        "effect": "max_health += 25; health = min(health + 25, max_health)"
    },
    {
        "name": "Stronger Bullets",
        "description": "Bullet damage x1.2",
        "effect": "damage_multiplier *= 1.2"
    },
    {

```

```

        "name": "Faster Trigger",
        "description": "Shoot faster",
        "effect": "fire_cooldown *= 0.85"
    },
    {
        "name": "Runner's Boots",
        "description": "Move faster",
        "effect": "speed *= 1.1"
    },
    {
        "name": "Spring Legs",
        "description": "Jump higher",
        "effect": "jump_strength *= 1.08"
    },
    {
        "name": "Quick Rounds",
        "description": "Bullets travel faster",
        "effect": "bullet_speed *= 1.15"
    },
    {
        "name": "Medkit",
        "description": "Heal 50 HP",
        "effect": "health = min(health + 50, max_health)"
    },
    ],
]

```

Do not add piercing bullets, lifesteal, knockback, double jump, shotgun, or multishot in version 1.

16. Required Classes

Implement these classes:

```

Game
Player
Bullet
Enemy
WalkerZombie
TankZombie
FlyingZombie
Platform
LevelManager
UpgradeManager
UI helpers

```

Responsibilities:

```

Game:
    Main loop, state machine, event handling, update and draw dispatch.

```

Player:
Movement, jumping, shooting, health, hurt timer, upgrades.

Bullet:
Movement, enemy collision, despawn outside screen.

Enemy:
Shared zombie health, damage, drawing, take_damage.

WalkerZombie:
Ground chase movement, platform collision, jumping.

TankZombie:
Same as Walker but slower, tankier, more contact damage.

FlyingZombie:
Direct movement toward player, ignores gravity and platforms.

Platform:
Static rectangle platform.

LevelManager:
Reads LEVELS, staggers enemy spawning, detects wave completion.

UpgradeManager:
Picks three upgrades, draws cards, applies selected upgrade.

UI:
Health bar, level text, menus, pause, game over, victory.

17. Asset Loading Behaviour

Implement safe image loading.

If an image exists, load it.

If the image does not exist, draw a rectangle fallback.

Suggested sizes:

```
player: 40 x 60
walker_zombie: 40 x 60
tank_zombie: 60 x 80
flying_zombie: 40 x 40
bullet: 12 x 6 or small circle
platform tile: optional
```

Never depend on image pixels for collision. Collision is always `pygame.Rect`.

18. MVP Coding Order

Build in this exact order:

Phase 1 — Window and Arena

```
Create Pygame window.  
Draw background.  
Draw ground and platforms as rectangles.  
Lock to 60 FPS.
```

Phase 2 — Player Movement

```
Create player rectangle.  
Move left and right.  
Add gravity.  
Add platform collision.  
Add jumping.  
Clamp player to screen.
```

Phase 3 — Shooting

```
Aim with mouse.  
Shoot bullets toward mouse.  
Normalize bullet direction.  
Apply fire cooldown.  
Despawn bullets off-screen.
```

Phase 4 — First Enemy

```
Create one Walker Zombie.  
Make it move toward player.  
Add platform collision for Walker.  
Make bullets damage Walker.  
Remove Walker when HP <= 0.
```

Phase 5 — Combat Loop

```
Add player health.  
Walker contact deals 10 HP.  
Add 0.5 second hurt invincibility.  
Add health bar.  
Add game over and restart.
```

Phase 6 — Level System

```
Add LevelManager.  
Spawn level 1 wave: 3 Walkers.  
Use staggered spawning.  
Detect wave clear.  
Show upgrade screen.  
Apply selected upgrade.  
Start next level.  
Add all 10 levels.
```

Phase 7 — Remaining Enemies

```
Add Tank Zombie.  
Add Flying Zombie.  
Confirm Tank stats: 100 HP, 25 contact damage, speed 1.0.  
Confirm Flying stats: 30 HP, 8 contact damage, speed 2.6, ignores platforms.
```

Phase 8 — Visuals and Polish

```
Add main menu.  
Add pause screen.  
Add victory screen.  
Add rectangle damage flashes.  
Add PNG images only after rectangle gameplay works.  
Add sounds only after the full loop works.
```

19. Scope Guard

Do not implement these in version 1:

```
Boss fights  
Multiple maps  
Scrolling camera  
Tile map editor  
Complex A* pathfinding  
Enemy projectiles  
Weapon switching  
Save files  
Animated sprites  
Lifesteal  
Piercing bullets  
Knockback bullets  
Double jump
```

Shotgun/multishot
Online multiplayer

20. Definition of Done

Version 1 is complete only when:

The player can complete levels 1-10.
Each level spawns the correct enemy counts.
Walker, Tank, and Flying Zombies all work correctly.
Zombies only damage by contact.
Enemies do not collide with each other.
Walker and Tank can jump and collide with platforms.
Flying Zombies pass through platforms and ignore gravity.
Base bullet damage is 10 HP.
Upgrade screen appears after every level.
All 7 upgrades work.
Upgrades stack correctly.
Player can die and restart cleanly.
Clearing level 10 shows victory screen.
The game is readable with rectangles.
Static PNGs can replace rectangles without changing collision logic.

Start with Phase 1. Do not skip ahead.